QUESTION-9

Stacked Using Linked List

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
struct Node *top = NULL;
void push(int value) {
    struct Node *newNode;
    newNode = (struct Node *)malloc(sizeof
        (struct Node));
    if (newNode == NULL) {
        printf("Stack Overflow\n");
        return;
    }
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("%d pushed into stack\n", value);
}
void pop() {
    struct Node *temp;
    if (top == NULL) {
        printf("Stack Underflow\n");
```

Output

```
Original List:
Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
After Deletion at Beginning:
Linked List: 20 -> 30 -> 40 -> 50 -> NULL
After Deletion at End:
Linked List: 20 -> 30 -> 40 -> NULL
After Deletion at Position 2:
Linked List: 20 -> 40 -> NULL

=== Code Execution Successful ===
```

main.c

```c
    temp = top;
    printf("Deleted Element: %d\n", top->data
        );
    top = top->next;
    free(temp);
}
void peek() {
    if (top == NULL)
        printf("Stack is Empty\n");
    else
        printf("Top Element: %d\n", top->data
            );
}
void display() {
    struct Node *temp;
    if (top == NULL) {
        printf("Stack is Empty\n");
        return;
    }
    temp = top;
    printf("Stack Elements:\n");
    while (temp != NULL) {
        printf("%d\n", temp->data);
        temp = temp->next;
```

Output

```
Original List:
Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
After Deletion at Beginning:
Linked List: 20 -> 30 -> 40 -> 50 -> NULL
After Deletion at End:
Linked List: 20 -> 30 -> 40 -> NULL
After Deletion at Position 2:
Linked List: 20 -> 40 -> NULL

=== Code Execution Successful ===
```

main.c

```c
}
void display() {
    struct Node *temp;
    if (top == NULL) {
        printf("Stack is Empty\n");
        return;
    }
    temp = top;
    printf("Stack Elements:\n");
    while (temp != NULL) {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}
int main() {
    push(10);
    push(20);
    push(30);
    display();
    peek();
    pop();
    display();
    return 0;
}
```

Output

```
Original List:
Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
After Deletion at Beginning:
Linked List: 20 -> 30 -> 40 -> 50 -> NULL
After Deletion at End:
Linked List: 20 -> 30 -> 40 -> NULL
After Deletion at Position 2:
Linked List: 20 -> 40 -> NULL

=== Code Execution Successful ===
```